CACHING — COMPLETE MASTER DOCUMENTATION

(Beginner → Senior Engineer → Production System Design)

---

0. HOW TO READ THIS DOCUMENT

This document is written to: - Build **intuition first** - Then explain **trade-offs** - Then show **real production patterns**

If you understand this fully, you will: - Design caching correctly - Answer **any caching interview question** - Avoid data inconsistency bugs in production

---

1. WHAT IS CACHING (CORE IDEA)

Caching is storing **copies of data** in a **faster storage layer** so that future requests can be served quickly **without hitting the primary data source**.

> 🔑**Golden Rule:** Cache is an optimization, **never the source of truth**.

Source of truth = **Database**

---

2. WHY CACHING EXISTS (PROBLEM IT SOLVES)

Without cache: - Every request → DB - High latency - DB overload - Poor scalability

With cache: - Faster responses - Fewer DB queries - Lower infra cost - Horizontal scalability

---

3. CACHING LAYERS (MOST IMPORTANT MENTAL MODEL)

```
CLIENT CACHE
(Browser / Mobile)
      ↓
SERVER CACHE
(Redis / In-memory)
      ↓
DATABASE CACHE
(Indexes / Buffer Pool)
```

```
        ↓
DATABASE STORAGE
```

Each layer has a **different responsibility**.

---

4. CLIENT-SIDE CACHING (WHEN & WHY)

4.1 What is Client Cache

- Browser HTTP cache
- CDN cache
- Service workers
- LocalStorage / IndexedDB

4.2 When to Use

🔗Read-heavy data 🔗Public or semi-public data 🔗Same response reused

4.3 When NOT to Use

🔩Sensitive data 🔩Financial / auth-critical data 🔩Real-time data

4.4 Examples

- Product listing
- Images, JS, CSS
- User profile (limited TTL)

4.5 HTTP Cache Example

```
Cache-Control: public, max-age=300
ETag: "v1.0"
```

Browser serves data without server call.

---

5. SERVER-SIDE CACHING (MOST CRITICAL LAYER)

5.1 What is Server Cache

- Redis / Memcached
- Shared across app instances
- In-memory → very fast

5.2 Why Redis

- Centralized
- TTL support
- Eviction policies
- High availability

5.3 Use Cases

- User profiles
- Dashboards
- Aggregations
- Permissions
- Feature flags

---

6. DATABASE-LEVEL CACHING

6.1 What it is

- DB buffer cache
- Query plan cache
- Indexes

6.2 What it Solves

- Disk I/O reduction
- Faster query execution

6.3 What it CANNOT Do

🔩Cannot reduce DB CPU load enough 🔩Cannot replace Redis

---

7. CACHE WRITE STRATEGIES (HEART OF CACHING)

7.1 CACHE-ASIDE (LAZY LOADING) ⭐DEFAULT

Read Flow

1. App → Cache
2. Cache miss → DB
3. DB result → Cache
4. Return response

Write Flow

1. App → DB
2. Cache DELETE (invalidate)

Properties

- DB is source of truth
- Eventual consistency
- High availability
- Simple & safe

When to Use

🔗80–90% systems 🔗User data 🔗Content systems

7.2 WRITE-THROUGH CACHE

Concept

Every write goes to **cache + DB synchronously**.

Properties

- Strong consistency
- Higher write latency
- Cache becomes critical dependency

When to Use

- Feature flags
- Auth configuration
- Pricing rules

7.3 WRITE-BEHIND (WRITE-BACK) CACHE

Concept

- Write only to cache
- DB updated asynchronously

Properties

- Very fast writes
- Risk of data loss

- DB lag

When to Use

- Logs
- Metrics
- Analytics

🔩Never for user-critical data

---

8. CONSISTENCY VS AVAILABILITY (CAP THEOREM)

CAP Theorem

You can choose only 2: - Consistency - Availability - Partition tolerance

Partition tolerance is mandatory → trade-off is:

Cache-Aside → AP

- System always responds
- Data eventually consistent

Write-Through → CP

- Data always correct
- System may reject requests

  Most real systems choose **Availability over Consistency**.

---

9. HOW TO KEEP CACHE CONSISTENT (REAL TECHNIQUES)

9.1 Invalidate on Write (BEST)

```
UPDATE DB
DELETE cache key
```

Why DELETE > UPDATE: - Prevent race conditions - Avoid partial writes

---

9.2 TTL (Time To Live)

TTL ensures cache self-heals.

Typical TTLs: | Data | TTL | |----|----| Profile | 5–15 min | Permissions | 1–5 min | Dashboard | 30–120 sec | Config | 10–60 min |

9.3 Versioned Keys

```
user:v2:123
```

Allows safe schema/data changes.

9.4 Cache Stampede Prevention

Problem: - Cache expires - 1000 requests hit DB

Solutions: - Randomized TTL - Mutex locks - Early refresh

---

10. EVICTION STRATEGIES (VERY IMPORTANT)

Eviction = what happens when cache is full.

Common Policies

Policy	Meaning	Use Case
LRU	Least Recently Used	Default, general
LFU	Least Frequently Used	Skewed access
FIFO	First In First Out	Simple
TTL	Time based	Time-sensitive

Redis Default

- LRU / LFU based

---

11. MULTI-LAYER CACHING (REAL PRODUCTION)

Example: Product Page

```
Browser Cache
  ↓
```

```
CDN Cache
  ↓
Redis Cache
  ↓
DB Index
  ↓
DB
```

Each layer reduces pressure on next.

---

12. PRODUCTION CODE PATTERN (CACHE-ASIDE)

READ

```
1. Check Redis
2. If hit → return
3. If miss → DB → set Redis
```

WRITE

```
1. Write DB
2. Delete Redis key
```

This pattern dominates industry.

---

13. WHEN TO USE WHAT (DECISION FRAMEWORK)

Ask in order: 1. Is data critical? → No cache 2. Read-heavy? → Redis 3. Public/static? → Client cache 4. Slow query? → DB index 5. Needs freshness? → Short TTL

---

14. COMMON MISTAKES (AVOID THESE)

🔩Treating cache as DB 🔩Updating cache blindly 🔩No TTL 🔩Caching transactional data 🔩No eviction planning

---

15. INTERVIEW-READY SUMMARY

> Caching improves performance and scalability by storing frequently accessed data closer to the application. Cache-aside is the most widely used strategy because it balances availability and consistency while keeping the database as the source of truth. Write-through is used when strong consistency is required, while write-behind is reserved for non-critical high-throughput systems. Effective caching requires careful eviction, TTL management, and consistency trade-offs guided by CAP theorem.

---

16. FINAL GOLDEN RULES (MEMORIZE)

1. Cache is never source of truth
2. Invalidate on write
3. TTL is mandatory
4. Prefer availability unless critical
5. Different data → different cache strategy

🔗This document represents **complete caching mastery equivalent to a senior backend/system design engineer**.